RideFlow: Detailed Guide to the Remaining Three Launch Steps

Repository: https://github.com/raiven427/ride-flow
Guide version: August 2026
Author: Manus AI

> This guide covers the three practical steps remaining before a controlled RideFlow pilot: authenticated browser verification; Stripe and M-Pesa sandbox testing; and the realtime GPS/dispatch plus activity-retention decision. It is an implementation and verification plan, not a declaration that the service is ready for unsupervised public transport operations.

Executive summary

RideFlow's source is portable and the current build passes typecheck, 18 Vitest tests, and production compilation. The preview can be inspected without login, but protected customer, driver, and admin operations require the owner to enter credentials directly in the browser. The payment code currently provides environment-driven Stripe Connect and Daraja scaffolding, protected procedures, and callback contracts; it still requires your provider credentials and sandbox verification. The current presence dashboard uses authenticated heartbeats and database activity events; live GPS dispatch requires a dedicated realtime transport and additional trip-state controls.

Step	Outcome	Owner input required
1. Authenticated verification	Prove customer, driver, and admin flows work with a real session	Complete OAuth login manually in the browser
2. Sandbox payments	Prove Stripe and M-Pesa request, callback, signature, and ledger paths	Create sandbox apps and enter secrets through the hosting secret manager
3. Realtime launch readiness	Choose a WebSocket/realtime service, implement dispatch states, and approve retention	Choose managed realtime versus self-hosted persistent service

Step 1: Authenticated browser verification

Why this step is manual

RideFlow uses an OIDC/OAuth session. The browser must receive the identity-provider cookie and callback state. For security, the owner should enter credentials directly in the open browser rather than sending them in chat, committing them to GitHub, or placing them in frontend code. If a password has ever been shared in chat, rotate it before production.

Before testing

1. Start the application on Ubuntu with `pnpm dev` or open the private preview URL.
2. Confirm the database is reachable and that the expected migrations have been applied.
3. Confirm the identity-provider redirect URI exactly matches the preview or deployment URL.
4. Confirm `DATABASE_URL`, `JWT_SECRET`, OIDC variables, and storage configuration are present in the deployment secret manager.
5. Open a private browser window or clear the existing RideFlow session so the test begins from a known state.

Customer test script

Open RideFlow and choose **Sign up** or the configured sign-in entry point. Select the rider role, complete the identity-provider login, and return to the application. Confirm that the header displays the signed-in identity and that the browser no longer shows the protected-action fallback.

Open **Book a ride**. Confirm that the route preview loads, a driver can be selected, ride preferences can be changed, women-only selection is visible where the product policy permits it, and the fare breakdown includes the base/distance/time amount, safety fee, 5% platform commission, and total. Click **Review ride** and confirm that the server-side quote procedure succeeds. Check the database for a quote row and its append-only ledger entries; do not treat a browser-only amount as authoritative.

Open **Profile** and choose a payment method. The current selector offers Cash, M-Pesa, PayPal, and Cards. In this stage it is a preference control; provider payment details should not be collected until the corresponding provider is configured. Test that selecting each option produces visible selection feedback without exposing payment credentials.

Choose a small profile image and upload it. Confirm the server validates the MIME type and size, stores the bytes through the configured object-storage adapter, persists metadata, and returns a success state. Do not upload real identity documents during a casual test.

Driver test script

Sign in with a separate test account, create or update a driver profile, and upload sandbox documents only. Confirm that license, insurance, and vehicle-document uploads are protected, accepted file types are enforced, and review status is persisted as pending. Confirm the driver dashboard begins with truthful zero values and no invented requests.

Test the driver availability control. Until dispatch is implemented, the dashboard should not imply that a real rider is being matched merely because a visual “nearby” cue exists. Replace or label any simulated proximity signal before inviting real drivers.

Admin test script

Sign in with the current admin account. Confirm that **Admin operations** appears only for the admin role. Open it and confirm the page distinguishes loading, failure, and empty states. Confirm the account list, online count, last-seen freshness, current view, and activity feed load from the protected procedure.

Open the Admin Control Room. Change a non-critical sandbox fare value and confirm that a new server-side quote uses the updated rule while historical quote and ledger rows remain unchanged. Change the notification recipient only to a controlled test mailbox. Test admin ownership transfer only with a verified second account, then verify that the old account loses admin access and the new account gains it.

Evidence and failure handling

Record only non-sensitive evidence: timestamp, test account label, URL environment, HTTP status, quote ID, provider transaction ID, and visible result. Never record passwords, session cookies, OIDC tokens, card security codes, M-Pesa PINs, raw identity files, or full webhook secrets.

Symptom	Likely area	First check
Redirect loops back to RideFlow	OIDC redirect/cookie configuration	Redirect URI, secure cookie, clock, and provider client settings
“Please sign in” after successful login	Session cookie or callback exchange	Browser cookie, server logs, JWT secret, callback response
Quote is displayed but not saved	Database or protected procedure	`DATABASE_URL`, migration state, server logs, quote table
Upload succeeds visually but no file metadata exists	Storage/database boundary	S3 credentials, bucket policy, file table, upload response
Admin navigation is visible to a non-admin	Frontend role condition	Server `adminProcedure` must still reject direct access
Admin dashboard appears empty forever	Presence/activity writes	Heartbeat mutation, database tables, last-seen timestamps

Step 2: Stripe sandbox and M-Pesa Daraja testing

Payment safety rules

Use test or sandbox credentials until the full flow is reconciled. Stripe states that sandboxes simulate objects without moving real money and that test and live objects are isolated [1]. Stripe also recommends storing keys in a secrets vault or environment variables rather than source control [1]. Daraja’s M-Pesa Express simulator uses a sandbox endpoint and asynchronous callbacks; the initial response only acknowledges that the request was accepted, while the callback carries the final result [2].

Stripe setup

Create or select a Stripe sandbox. Obtain the sandbox secret key, publishable key if the frontend requires one, Connect account settings if driver payouts will be tested, and webhook signing secret. Add them through the hosting secret manager, not by editing committed files.

The RideFlow payment scaffolding expects environment-driven configuration. Inspect the repository’s environment template and payment module for the exact variable names before setting them. Do not guess variable names or place `sk_test_` values in `client/src`. Keep the server secret only on the backend.

Configure a webhook endpoint on the deployed HTTPS URL. The handler must receive the raw request body, verify the `Stripe-Signature` header with the endpoint secret, reject invalid signatures, return a fast 2xx response after safe acceptance, and process recognized events idempotently. Stripe's official webhook guidance explicitly recommends signature verification and quick successful responses before complex work [3].

For local testing, use the Stripe CLI according to Stripe's current documentation [3]. Forward sandbox events to the local endpoint, trigger a test event, and compare the event ID with the stored idempotency record. Then test the deployed HTTPS endpoint separately; a successful local CLI forward does not prove that production routing, TLS, secrets, or firewall rules are correct.

Stripe test matrix

Case	Action	Expected result
Successful card payment	Use an official Stripe sandbox test card and future expiry	Payment intent succeeds; ledger is reconciled once
Declined card	Use Stripe's documented decline test value	Payment is marked failed; no driver payout or completed ledger entry
Duplicate webhook	Deliver the same event twice	One business transition; second delivery is safely ignored
Invalid signature	Alter the signature or use the wrong secret	HTTP 400/unauthorized handling; no ledger mutation
Connect transfer readiness	Use sandbox connected-account identifiers	Platform fee and driver amount are separated; no live transfer occurs
Provider timeout	Delay or fail the handler dependency	Event is retried or placed in a recoverable state; no duplicate charge

M-Pesa Daraja setup

Create a Daraja developer account and a sandbox app in Safaricom's developer portal. The official M-Pesa Express simulator identifies the sandbox STK Push endpoint, requires consumer credentials, shortcode/passkey test data, a callback URL, and a valid test phone number format

[2]. Use the exact values and endpoint shown in the current portal rather than copying production values into sandbox configuration.

Configure the server with the Daraja consumer key, consumer secret, business shortcode, passkey, callback URL, environment, and any required timeout settings. The callback URL must be publicly reachable over HTTPS in a deployed environment. For local development, use a secure tunnel only for a temporary sandbox test and never expose production secrets through that tunnel.

When RideFlow sends an STK Push, store a pending payment record with the local ride or quote identifier, MerchantRequestID, CheckoutRequestID, amount, and status `pending`. Do not mark the ride paid from the initial acknowledgment. Wait for the callback, verify that the callback identifiers match the pending request, parse `ResultCode`, and mark success only when the result code indicates success. Store the receipt number when present and make callback handling idempotent.

M-Pesa test matrix

Case	Action	Expected result
Accepted STK request	Submit a sandbox request with valid test data	Store pending state and provider request IDs
Successful callback	Complete the simulator flow	Store success, receipt, amount, and customer phone metadata as permitted
Cancelled callback	Cancel the prompt	Store a failed/cancelled state; do not complete the ride
Duplicate callback	Deliver the same callback twice	One state transition and one ledger effect
Wrong callback ID	Send a callback for another request	Reject or quarantine it; no ride mutation
Invalid credentials	Use a deliberately invalid sandbox secret in staging	Clear configuration error; no partial payment state
Callback unavailable	Temporarily stop the callback endpoint	Pending payment remains recoverable and can be reconciled

Reconciliation requirements

A payment provider response is not the same thing as a completed RideFlow trip. Define a transition table linking `payment_pending`, `payment_succeeded`, `payment_failed`, `ride_completed`, `refund_pending`, and `refunded`. Reconcile provider events against RideFlow's append-only ledger using provider IDs and idempotency keys. Never calculate the 5% commission from a client-provided total. The server should calculate the fare and write the rider charge, driver earning, and platform commission as separate ledger entries.

Step 3: Realtime GPS, dispatch hosting, and activity retention

Choose the realtime architecture

WebSockets provide a two-way browser/server communication session without repeated polling [4]. For an initial pilot, choose one of these approaches:

Approach	Advantages	Tradeoffs	When to choose
Managed realtime provider	Faster implementation, presence primitives, scaling handled by vendor	Vendor cost, data-processing review, provider lock-in	Small team prioritizing launch speed
Self-hosted WebSocket service	Full control, portable protocol, predictable data boundary	Persistent hosting, reconnect logic, monitoring, scaling, incident response	Team prepared to operate realtime infrastructure
Short polling fallback	Simplest first prototype	Higher latency, more requests, poor GPS experience, not suitable for live dispatch	Only for a temporary internal demo

For continuous or sub-minute tracking, do not use a low-frequency AI schedule. Use a persistent realtime service or an appropriate managed provider. A WebSocket connection should be closed when the page is finished to avoid stale sessions and browser lifecycle issues [4].

Minimum trip-state model

Implement a server-owned state machine before exposing real dispatch:

```
requested
  → matching
  → driver_assigned
  → driver_arriving
  → driver_waiting
  → in_progress
  → completed

Any active state → cancelled
completed → disputed or refunded through controlled procedures
```

Every transition must include the actor, timestamp, previous state, new state, and an idempotency key. The server must reject impossible transitions, such as `completed → driver_arriving`. Dispatch must be idempotent so retries cannot assign multiple drivers or create duplicate trips.

GPS data flow

The driver device sends location updates only while the driver is online or on an active trip. The server authenticates the connection, verifies the driver owns the active trip, validates coordinates and timestamps, applies rate limits, and publishes only the minimum location precision needed by each authorized viewer. The rider should see the assigned driver; an unrelated account should see nothing.

Store the latest location in a mutable current-location record and store only selected historical points for operational, safety, or legal needs. Do not put every GPS point into the generic activity feed. Activity events should record operational facts such as assignment, pickup, cancellation, and completion, not an unbounded location stream.

Dispatch test cases

Case	Expected result
Two eligible drivers request the same ride	Exactly one assignment succeeds; the other receives a clear outcome
Driver disconnects	Rider sees stale/connection state; dispatch can reassign according to policy
GPS jumps outside plausible speed	Update is rejected or flagged, not blindly shown as truth
Rider opens another account's trip URL	Server returns forbidden/no data
Client retries assignment	Same trip and assignment result, no duplicate driver assignment
Trip is cancelled during matching	Matching stops and late driver responses are ignored
Realtime service restarts	Clients reconnect with backoff and reload authoritative trip state

Hosting decision

A normal autoscaling request/response host is sufficient for most API and dashboard work, but a persistent WebSocket worker or always-on realtime process needs a hosting mode that supports long-lived connections. If the selected managed provider supplies the realtime transport, RideFlow can keep the main API stateless. If RideFlow self-hosts WebSockets, use persistent hosting, health checks, connection metrics, graceful shutdown, and a shared broker when more than one application instance is introduced.

Do not choose a cloud computer merely because the application has realtime requirements. First compare a managed realtime service with persistent hosting that supports WebSockets. Choose a more powerful server only when the workload needs custom runtimes, fixed network control, or resources beyond the application host's limits.

Activity-retention policy

Approve a written policy before enabling automated deletion. A reasonable starting point is 90 days for detailed operational activity, 12 months for security-sensitive admin events, and shorter

retention for raw GPS traces unless a documented safety or legal need exists. Confirm the final durations with qualified local privacy and transport advisers.

Add an index on the event creation timestamp. Run cleanup in bounded batches, record the number of deleted rows and duration, and alert on failure. Test the query in staging, take a backup, and verify that recent events remain. Never run a broad destructive cleanup without a recovery plan.

A daily deterministic cleanup job is appropriate for retention. It should not invoke an AI session for a simple database delete. If the application host provides a managed heartbeat/cron facility, run the cleanup there. If realtime and cleanup share a persistent worker, keep them as separate jobs with separate metrics and failure handling.

Ubuntu command checklist

```
cd /home/ubuntu/rideflow
pnpm install
cp .env.example .env
chmod 600 .env
nano .env
pnpm drizzle-kit generate
pnpm drizzle-kit migrate
pnpm check
pnpm test
pnpm build
pnpm dev
```

Before any production secret change, create a backup of the secret values in the provider’s secure vault, not in a Git repository. Before any schema change, test the migration on a disposable database. Before any payment test, verify that the environment is sandbox mode.

Definition of done

The three remaining steps are complete only when the owner has a successful signed-in browser run, a passing Stripe and Daraja sandbox matrix with verified callbacks and idempotent ledger effects, and an approved realtime/retention design with a staging test. A green frontend build alone is not sufficient evidence of payment or dispatch readiness.

References

[1]: https://docs.stripe.com/testing-use-cases Stripe, “Testing use cases.”

[2]: https://developer.safaricom.co.ke/apis/MpesaExpressSimulate Safaricom Daraja, “M-Pesa Express Simulate.”

[3]: https://docs.stripe.com/webhooks Stripe, “Receive Stripe events in your webhook endpoint.”

[4]: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API MDN Web Docs, “WebSocket API.”